ForecastLabAI — PydanticAI agents with human approval

Problem

Explain the agentic layer: what the agents can do, how they are bounded, and why it is safe enough.

Instant solution

Two agents (`app/features/agents/agents/`)

- **Experiment orchestrator** (`experiment.py`), tools: `tool_list_runs` , `tool_get_run` , `tool_run_backtest` , `tool_compare_backtest_results` , `tool_compare_runs` , `tool_create_alias` , `tool_archive_run` , `tool_propose_scenario` , `tool_save_scenario` . Output: `ExperimentReport` (run_id, status, summary, metrics, recommendations).
- **RAG assistant** (`rag_assistant.py`), tools: `tool_retrieve_context` , `tool_format_citations` , `tool_check_evidence` , `tool_list_sources` . Output: `RAGAnswer` (answer, confidence, sources, no_evidence).

Four principles

1. **Tool calling** — the LLM picks from registered tools that call existing services; it never runs arbitrary code.
2. **Dependency injection** — `AgentDeps` (`deps.py`) gives tools the DB session, session ID, request ID and tool-call counter.
3. **Structured output** — `PromptedOutput(ExperimentReport | RAGAnswer)` : the app receives validated Pydantic objects.
4. **Human-in-the-loop** — `agent_require_approval = [create_alias, archive_run, save_scenario]` (configurable action keys). These create a pending action; the service layer executes only after approval.

Session TTL 120 min (enforced). Streaming via WebSocket `/agents/stream` .

Interview answer

> *The LLM is not an unrestricted decision-maker. It gets typed tools wired to existing services; the backend validates and executes. Sensitive state changes like moving a deployment alias need human approval. That gives useful automation with explicit control.*

Why it works / gotchas

- Agent vs chatbot: an agent chooses tools, inspects results and takes bounded steps toward a goal.
- **Don't claim a tool-call cap.** `agent_max_tool_calls = 10` and `agent_max_sessions_per_user = 5` exist in `app/core/config.py` , but nothing enforces them: the counter only increments and `agent.run` gets no usage limits (and there is no auth, so no "user"). Good answer to "what would you add?": enforce them via PydanticAI usage limits.
- Models are `provider:model` strings (default `anthropic:claude-sonnet-4-5` , fallback `openai:gpt-4o`); swappable from the AI Models Console.

Diagram brief

- Type: flowchart with a decision diamond.
- User request → PydanticAI agent (system prompt · model · output schema) → Tool call (AgentDeps: DB session, session/request ID, counter).
- Diamond "approval required?" — No → execute tool → result back to agent; Yes → pending action → human approves/rejects → service layer executes.
- End: validated `ExperimentReport` / `RAGAnswer` → WebSocket → frontend. Do NOT draw a tool-call cap (configured, not enforced).